HW3 DOCUMENTATION - MUHAMMET FURKAN YILGIN - 220104004964

Expression Evaluation Strategy:

Terminals:

The terminals represent the basic units of the language, such as operators (+, -, *, /), keywords (KW_EXIT, KW_DEF), identifiers (IDENTIFIERS) and fraction numbers(VALUEFS).

Non-terminals:

The non-terminals represent higher-level structures in the language, such as expressions (EXP) and functions (FUNCTION).

Rules:

The rules define how expressions are constructed and how functions are defined. For example, an expression can be a binary operation (OP_PLUS, OP_MINUS, OP_MULT, OP_DIV) applied to two VALUEFs.

Semantic Actions:

The semantic actions embedded in the rules perform specific actions during parsing, such as performing arithmetic operations or identifying function definitions.

Numerical Representation:

Fraction Representation: Given that expressions always return a fraction, it's essential to represent numbers as fractions, where each fraction is represented as a pair of integers between the letter 'f' (numerator(integer before the letter f) and denominator(integer after the letter f)).Also it is important that the condition of denominator being zero should be handled for error handling and appropriate evaluation of expressions.

Simplification:

Simplifying Fractions: A operation to simplify fractions using the gcd of two fraction numbers after each operation must be implemented so that the result of the expression is in the simplest form.

Numerical Operations:

Operations on Fractions: Addition and substraction of 2 VALUEF expressions(fraction values) must be done by equalizing the denominators and doing the operation on numerators and then simplifying the final result.However, multiplication must be done by directly multiplying and simplifying and division must be performed by multiplying the first VALUEF with second reverse VALUEF fraction and simplifying.

Functions :

- Parameter passing by value.(call by value)
- Function definition returns a function value
- Function application will return the value of the expression evaluated

Scope:

Function Scope:

Local Variables: Functions have local variables with identifiers and valuefs as parameters and global functions to calculate the valuefs with the given identifier name.Variables defined outside any function have global scope.

Global variables are accessible throughout the program.

Function Definition and Application:

Parameter Passing: Parameters are passed by value in function definitions or returns the result of two fraction values or the value of the expression evaluated with the identifier name.

Identifier Scope:

Variable Scope: The language supports variables (identifiers), and the parser can handle identifier expressions by calling the appropriate function matches with the CFG.

Error Handling: Division by zero for denominator being zero and the other syntaxs that is not included in the given context free grammar.

Conflict Resolutions:

Operator Precedence: Multiplication and division have higher precedence than addition and subtraction.

Control Flow:

Control flow constructs (e.g., conditionals and loops) msut be added to enhance the language's expressiveness.Such as if,for,while conditions etc.

Data Types:

Support for different data types, such as lists,strings,boolean values or etc.

- Also the Context Free Grammar that has been shared with us does not consist of some tokens that were in the previous homework(HW2) and in the lexer that we created.These tokens are :

KW_AND, KW_OR, KW_ NOT, KW_EQUAL, KW_LESS, KW_NIL, KW_LIST, KW_APPEND, KW_CONCAT, KW_SET, KW_FOR, KW_IF, KW_LOAD, KW_DISPLAY, KW_TRUE, KW_FALSE and COMMENT. To fix any conflict resolutins or etc. we can use the context free grammar below that consists all of the tokens above.

Also in the context free grammar shared with us the OP_CP(“)”) is missing for some expressions too so I added them to the end of some expressions too.

$EXP -> OP_OP OP_PLUS $EXP $EXP OP_CP

| OP_OP OP_MINUS $EXP $EXP OP_CP

| OP_OP OP_MULT $EXP $EXP OP_CP

| OP_OP OP_DIV $EXP $EXP OP_CP

| OP_OP IDENTIFIER $EXP OP_CP

| OP_OP IDENTIFIER $EXP $EXP OP_CP

| OP_OP IDENTIFIER $EXP $EXP $EXP OP_CP

| OP_OP KW_AND $EXP $EXP OP_CP

| OP_OP KW_OR $EXP $EXP OP_CP

| OP_OP KW_NOT $EXP $EXP OP_CP

| OP_OP KW_EQUAL $EXP $EXP OP_CP

| OP_OP KW_LESS $EXP $EXP OP_CP

| OP_OP KW_NIL $EXP $EXP OP_CP

| OP_OP KW_TRUE $EXP $EXP OP_CP

| OP_OP KW_FALSE $EXP $EXP OP_CP

| OP_OP KW_SET IDENTIFIER $EXP OP_CP

| OP_OP KW_FOR $EXP $EXP OP_CP

| OP_OP KW_IF $EXP $EXP OP_CP

| OP_OP KW_LOAD $EXP $EXP OP_CP

| OP_OP KW_DISPLAY $EXP $EXP OP_CP

| IDENTIFIER | VALUEF | LIST | COMMENT

KW_LESS | KW_EQUAL | KW_NOT | KW_OR | KW_AND // for the if condition expressions

For the expressions list,append and concat we need lists but i did not implement a new expression for them since in our CFG there weren’t any data type called lists.

OP_OP KW_LIST $EXP $EXP OP_CP

| OP_OP KW_APPEND $EXP $EXP OP_CP

| OP_OP KW_CONCAT $EXP $EXP OP_CP